

Второй этап заполнения отчета для ресурса "Домен: Отчет НИОКТР":

---Экспериментальные разработки---

***Наименование разработки**

Робототехнический, двухуровневый программно-технический комплекс "Horizon Automation".
Состоящий из:

- 1 - уровень программируемого логического контроллера ПЛК (контроллерный уровень), работающего под управлением соответствующего оригинального, запатентованного фреймворка "Horizon Automation PLC";
- 2 - уровень концентратора (серверный уровень), работающего под управлением соответствующего оригинального, запатентованного фреймворка "Horizon Automation Server".

***Научно-технический результат**

Конструкторская документация, программное обеспечение

***Уровень готовности технологий**

Уровень готовности технологии 95%

---Фактический результат---

***Фактический результат**

Разработан программно-технический комплекс, предназначенный для автоматизации широкого круга прикладных задач, организации разнообразных образовательных программ различного уровня учебных заведений. Запатентованы два РИД: фреймворк (программное обеспечение) "Horizon Automation PLC" и "Horizon Automation Server", полученные в результате работы.

***Описание результата**

Программно-технический комплекс "Horizon Automation" представляет собой высоко интегрируемую, многофункциональную платформу, работающую на двух "уровнях" автоматизации.

На первом – полевом уровне работают Программируемые Логические Контроллеры (ПЛК) под управлением оригинального, запатентованного фреймворка – "Horizon Automation PLC".

На втором – серверном уровне работают концентраторы (Hub), под управлением оригинального, запатентованного фреймворка – "Horizon Automation Server".

Двухуровневая система на основе ПЛК и Hub представляет собой комплекс, предназначенный для - автоматизированного сбора, управления, обработки, визуализации и хранения данных - сенсоров, актуаторов элементов промышленной автоматики, IoT систем, систем "умный дом", ПЛК и иных информационно-измерительных систем.

ПЛК полевого уровня могут работать в проектах в которых требуется только локальная автоматизация и в этом случае верхний (серверный) уровень не требуется, т.е. фактически

автономно. В таком случае ПЛК под управлением фреймворка "Horizon Automation PLC" становится многофункциональным контроллером способным собирать с десятков и даже сотен сенсоров данные, оперативно их обрабатывать в автоматическом режиме, при необходимости сохранять результат на встроенных ресурсах flash памяти контроллера или, если предусмотрено проектом, сохранять результаты в виде файлов microSD карте подключенной к ПЛК. Также фреймворк позволяет вести управление многообразными актуаторами (исполнительными устройствами), такими как моторы, шаговые двигатели, сервоприводы, средства индикации, запорная арматура и т.п.

Фреймворк полевого уровня – "Horizon Automation PLC" предназначен для установки на 32-разрядный программируемый логический контроллер с архитектурой ARM серии cortex M4 или старше либо Tensilica Xtensa LX6 / LX7. Данные архитектуры принадлежат, разрабатываются, производятся и поддерживаются разными производителями, что делает платформу очень устойчивой к негативным геополитическим факторам.

Исполнительная среда фреймворка – Espruino является open source продуктом. Все взаимодействие и разработка прикладного кода в рамках фреймворка ведется на языке JavaScript, с открытым стандартом.

Фреймворк вводит понятие источника и получателя данных. И то и другое в итоге конвертируется в измерительно-управляемый канал, что фактически означает создание цифровой копии физического сенсора, актуатора или логического внешнего источника. Все данные от всех источников и/или получателей проходят через математический тракт обработки, после чего преобразуются в цифровые измерительные каналы, с которыми работает пользователь.

Для создания сколь угодно сложных прикладных алгоритмов, пользователю предоставляется вся мощь современного стандарта языка JavaScript ECMA ES6/ ECMA ES7. Одно из уникальных возможностей предоставляемых фреймворком является подсистема REPL (read-eval-print loop — „чтение — вычисление — вывод“). Это позволяет подключиться к работающему ПЛК и в реальном времени получить от работающей системы те или иные данные, получение которых не предусмотрено разработанной прикладной программой. Подключение производится либо посредством USB кабеля, либо по сети по протоколу Telnet, Такие функции дают мощные инструменты отладки и даже корректировки действующей программы без перекомпиляции и пересборки программы. Данная возможность ускоряет отладку прикладных алгоритмов на порядок по отношению к классическим системам, а также позволяет строить невероятно гибкие образовательные комплексы для проведения занятий по курсам физики, математического моделирования, информатики, моделирования систем управления БПЛА.

Конфигурация фреймворка и настройка прикладного проекта выполняется также очень комфортно, а именно путем записывания конфигурационных текстовых, человека-читаемых файлов в формате JSON, который стал стандартом в области хранения конфигурационной информации в индустрии IT. Посредством механизма REPL, пользователь дистанционно имеет возможность как считать хранящуюся на ПЛК конфигурацию и прикладную программу так и записать новую – все это выполняется без пересборки системы. Еще одна уникальная особенность фреймворка – это возможность заранее записать несколько прикладных программ на ПЛК, и предоставить пользователю механизм выбора текущей исполняемой программы, в том числе полностью автоматический вариант, когда система в результате внутренних вычислений примет решение о необходимости перезапустить всю систему и стартовать другую, заранее приготовленную прикладную программу.

Для коммуникации с внешним миром фреймворк предусматривает развитые возможности. Базовым протоколом для соединения с серверным уровнем платформы является легкий и

быстрый оригинальный, разработанный в ходе работ над проектом протокол LHP, использующий стек протоколов TCP/IP и прикладной протокол WebSocket. Служба протокола работает в режиме сервера LHP и ожидает входящие соединения. Протокол реализует двухсторонний режим обмена во многом похожий работу известного MQTT брокера Mosquitto, но оптимизированный под скромные аппаратные возможности микроконтроллеров, на базе которых может работать фреймворк.

Другой возможностью получать данные от внешних источников и передавать данные во внешний мир, является служба фреймворка MQTTGateway. С ее помощью реализуется концепция виртуального сенсора или актуатора. Служба подключается к внешнему MQTT брокеру, и либо подписывается на топики брокера, либо публикует сгенерированные данные на топики брокера.

Все поступающие данные от брокера проходят обработку через унифицированный тракт математической обработки, и в итоге конвертируются в измерительные каналы, не отличимые от тех, что создаются на основе физически подключенных к ПЛК сенсоров и актуаторов. Таким образом пользователь работает в однородной среде, не зависимо от того как и откуда ему необходимо получить данные и передать результат обработки.

Для физического взаимодействия с внешним миром фреймворк предоставляет развитые механизмы коммуникации на базе сетей Wi-Fi или Ethernet, для чего в нем уже присутствует набор драйверов - коммуникационных шин. Для взаимодействия с современными цифровыми сенсорами и актуаторами, работающими по различным протоколам и использующие различные физические шины (I2C, UART, SPI, OneWire и др), во фреймворке присутствуют соответствующие драйвера. Для подключения новых сенсоров, актуаторов и логических источников, пользователю предоставляется продуманный набор классов JavaScript для реализации драйверов новых устройств, что позволяет неограниченно расширять прикладную платформу без участия производителя.

Фреймворк наделяет ПЛК сервисными функциями присущими профессиональным системам автоматизации, такие синхронизация астрономического времени платформы от внешнего NTP сервера, поддержку DHCP протокола, поддержку mDNS протокола позволяющего находить ПЛК в сети по его имени, перенаправление пользовательских отладочных сообщений стандартной системы логирования JavaScript - console.log на сервер логирования Enterprise уровня – Graylog, с высокой детализацией сообщений. Внешним системам доступны каналы с системными данными, такими как температура ядра микроконтроллера, значения системной памяти, количество занятых и оставшихся GPIO портов GPIO, значения системных часов др.

Фреймворк превращает ПЛК в функциональное средство построения полевого уровня систем промышленной автоматики, компонентов IoT систем, системы умный дом. Легко интегрируется с промышленными системами класса MES, APS, SCADA.

ПЛК на базе платформы "Horizon Automation PLC" идеально подходит для организации разнообразных образовательных программ для различного уровня учебных заведений, по курсам - физика, химия, информатика, математическое моделирование, робототехника, моделирование БПЛА.

Полные возможности ПЛК, работающего на основе фреймворка полевого уровня "Horizon Automation PLC" раскрываются во взаимодействии с фреймворком серверного уровня "Horizon Automation Server". В таком случае возможности комплекса на порядок увеличиваются. Более подробно такая схема построения комплекса показана при описании серверного фреймворка.

Фреймворк серверного уровня – "Horizon Automation Server" предназначен для установки на компактный 64-разрядный одноплатный компьютер с архитектурой ARM или x86 под

управлением ОС Linux. Таким образом фреймворк серверного уровня можно развернуть практически на любой аппаратной платформе класса микрокомпьютер, любого производителя. Что делает платформу очень устойчивой к негативным геополитическим факторам.

Исполнительная среда фреймворка построена на тандеме двух open source продуктов – Node.js. и Node-Red. Все взаимодействие пользователя и разработка прикладного кода в рамках фреймворка ведется на языке JavaScript в среде потокового программирования Node-Red. Разработку прикладного кода можно вести не только на языке JavaScript, но и на других популярных языках программирования, например Python.

Архитектура фреймворка реализована как набор микросервисов которые называются системными службами в терминологии фреймворка. Архитектура построения и взаимодействия напоминает "слоеный пирог". Верхний слой составляют службы: Source, Gateway, Server.

Службы типа Source – отвечают за взаимодействие с источниками данных, в роли которых могут выступать физические сенсоры, внешние ПЛК, сторонние информационно-измерительные системы. Именно данные службы предоставляют специализированные сервера и протоколы позволяющие использовать очень большое количество источников. Входные данные, полученные через эти службы в итоге, конвертируются в информационно-измерительные каналы, аналогичные тем, что создаются в фреймворке полевого уровня "Horizon Automation PLC". В итоге на серверном уровне также реализуется концепция цифровой копии сенсоров, актуаторов, логических источников. Так же, как и в фреймворке полевого уровня данные от всех источников проходят через математический тракт обработки, прежде чем становятся унифицированными каналами.

Службы типа Gateway – отвечают за маппирование ("зеркалирования") всех информационно-измерительных каналов порождаемые фреймворком на сторонние информационно-измерительные системы. Например, на внешний брокеры Mosquitto MQTT, Apache Kafka, OPC и др.

Службы типа Server – отвечают за взаимодействие с внешними информационно-измерительными системами, работающими по специализированным или закрытым протоколам, в случае если взаимодействие с внешними системами через унифицированные протоколы или брокеры не рационально или невозможно по техническим причинам.

Следующий слой составляют службы, связанные оперативным и долговременным хранением – провайдеры баз данных, отвечающие за сохранение поступающих от источников данных или генерируемых фреймворком в базе данных. Сохранение подвергаются как "сырые" данные, только что поступившие в Hub, до того, как они пройдут тракт математической обработки, так и данные каналов после того, как данные прошли полную обработку, заданную пользователем. Также провайдеры БД отвечают за хранение всей конфигурационной информации разворачиваемой системы автоматизации. В стандартной комплектации серверного фреймворка присутствуют два провайдера БД – NoSQL документо-ориентированной MongoDB и NoSQL структурированных хранилищ данных в памяти в Redis.

Следующий слой составляют службы логирования. Это важные компоненты системы. При работе системы порождаются и проживают свою жизнь множество информационных сигналов. Одни отражают жизненные циклы платформы, другие связаны с подключением/отключением источников, третьи возникают при взаимодействии с внешними информационно-измерительными системами. Даже при небольшом проекте автоматизации, за один час работы через системы могут проходить тысячи таких сигналов.

При запуск прикладной системы перед пользователем неизбежно встанет вопрос как запустить систему, как отследить корректность запуска всех служб, установления всех взаимосвязей, корректности работы источников и многого другого. Встанет вопрос – как отлаживать всю эту непростую, с точки зрения внутреннего устройства систему? Именно эти вопросы решают службы позволяющие пользователю вести тотальный учет любого сообщения любой из системных служб, всех фаз запуска/работы/завершения работы служб фреймворка. Основная служба, отвечающая за реализацию данного функционала – Logger. Данная служба обладает исключительно гибкими возможностями по настройке перехвата и логгирования сообщений, возникающих в системе. При этом данная служба фреймворка работает в тандеме с Enterprise продуктом в области логгирования Graylog, с включенной подсистемой полнотекстового поиска. При этом служба фреймворка Logger подготавливает детальный пакет в формате GELF (родной протокол Graylog) тем самым давая уникальные возможности поиска сообщений в журналах Graylog. Пакет устроен таким образом что, что можно с высокой скоростью и точностью отфильтровать лог-сообщение от конкретного микросервиса, с конкретным уровнем логирования и т.д. и т.п. Помимо возможности логирования всех сообщений ядра фреймворка, разумеется, проработано логирование прикладных сообщений от кода пользователя, размещённого в user space. Помимо развитого поиска Graylog предоставляет исключительно гибкие возможности по созданию и кастомизации дашбордов, на которых в самых разнообразных формах можно выводить данные конкретного сообщения или группы сообщений, выполняя различные виды предобработки (подсчет количество сообщений всего рот источника, в ед времени, макс значение, минимальное значение и т.д. и т.п.). Важная особенность всей системы логгирования разработанная входе проекта, заключается в том что, помимо тонкой настройки состава сообщений отправляемых в Graylog, он настроен таким образом что сообщения от ПЛК контроллеров, ядра серверного фреймворка, пользовательских сообщений user space, иных внешних служб и аппаратного обеспечения в нем разделены. Это сделано с помощью механизма отдельных Input-в в Graylog. В том числе все сообщения OS Linux перенаправляются на Graylog, а с учетом великолепного web интерфейса логгера мы имеем дистанционно доступный интерфейс просмотра всех сообщений от всех программных компонентов ПТК, от всех аппаратных компонентов ПТК, а также от всех аппаратных компонентов проекта в целом. Например, в составе проекта предусмотрены различные сетевые компоненты, такие как коммутаторы, маршрутизаторы, точки доступа, а также сторонние ПЛК, сервера, панели взаимодействия с оператором HMI и т.д. и т.п. Все лог-сообщения от данных источников можно перенаправить на систему логирования фреймворка и они будут записаны в отдельный Input системы логирования.

Следующий слой составляют службы, формирующие главную сущность платформы – информационно-измерительные каналы, которые являются цифровыми двойниками физических сенсоров, актуаторов, логических каналов. Все процессы, связанные с созданием таких каналов в соответствии с конфигурацией, жизненным циклом управляются данными службами. Именно данные службы создают основу для дальнейшей работы пользователей комплекса, которые будут разрабатывать прикладной проект.

Следующий слой составляют службы формирующие системные шины, это своего рода информационные магистрали, которые "склеивают" все службы в единый фреймворк, а также обеспечивают мост между ядром фреймворка и пространством пользователя, в котором рождается прикладной проект. Все шины построены на событийной модели сервера Node.js.

Следующий слой составляют службы, формирующие интерфейс между, собственно, фреймворком и user space уровнем, где работает пользователь. Эти службы определяют

правила взаимодействия между фреймворком и user space, причем здесь речь идет о взаимодействии на уровне улов потоковой среды Node-Red. Необходимо отметить, что ядро серверного фреймворка хотя технически работает в среде потокового программирования Node-Red, фактически минимально использует функционал Node-Red, большая часть функционала ядра фреймворка работает напрямую с сервером Node.js и только при переходе в user space пользователю предоставляются коннекты к стандартным узлам Node-Red. Такая схема позволяет совместить скорость и гибкость работы кода на базе среды исполнения Node.js и гибкость, лаконичность построения прикладного проекта в среде потокового программирования Node-Red. Также такая схема организации позволяет легко освоить комплекс так как в конечном итоге работа ведется в среде имеющей миллионное комьюнити по всему миру и что более важно, в России также присутствует большое количество разработчиков знакомых с данной системой и языком программирования JavaScript.

Следующий слой составляют службы, формирующие графический интерфейс комплекса на базе возможностей дашбордов Node-Red. Данные службы позволяют конфигурировать систему с использованием web сервера и набора графических виджетов Node-Red. Для проектов, в которых предъявляются более жесткие требования к графическим инструментам наличию профессиональной подсистемы алертов, метрик и т.д., фреймворк предлагает функционал данной службы, который позволяет использовать для отображения, вычисления метрик, службы алертов промышленный стандарт в данной области – Grafana.

Следующим уровнем серверного уровня Hub являются сторонние информационно измерительные системы, которые также являются важной частью ПТК. Выше, при описании системных служб были указаны сторонние инструменты, для которых фреймворк предоставляет коннекторы и драйвера для совместной работы. В ходе реализации проекта было принято решение о необходимости не только разработать интеграцию с данными инструментами, но и не посредственно встроить их в программное обеспечение серверного уровня. Провести кастомизацию и настройку на совместное использование. Данное решение сэкономит конечному пользователю много времени на компоновку всех продуктов в единое решение. Таким образом Hub уже из "коробки" содержит следующие компоненты, которые были развернуты, кастомизированы, подключены к системным службам фреймворка "Horizon Automation Server":

- * MongoDB – NoSQL документо-ориентированная СУБД;
- * Redis - NoSQL структурированных хранилищ данных в памяти СУБД;
- * Node.js – серверная исполнительная среда исполнения JavaScript;
- * Node-Red - среда потокового программирования и визуализации на языке JavaScript;
- * Graylog - сервер логгирования;
- * Grafana - среда визуализация;
- * Mosquitto - брокер MQTT;
- * MQTTX - web клиент MQTT;
- * RasAP - служба Wi-Fi "hot spot";
- * Traefik - реверс прокси;
- * Dnsmasq - DNS сервер;
- * NTP - сервер времени;

Помимо прикладных уровней, проектом предусмотрена разработка сервисных и общесистемных служб. При работе Hub, подключение к нему ПЛК происходит по протоколу DHCP, что снимает с пользователя строгое распределение IP адресов между элементами полевого уровня. Но в тоже время это приводит к тому что пользователь "не знает" как обратиться к конкретному ПЛК, как сконфигурировать работу источников которые работают с

данными ПЛК и иными источниками данных. Поэтому В рамках ПТК проработана связка служб DHCP Server + DNS + RasAP HotSpot. Интеграция с данными службами позволяет:

- динамически распределять IP адреса из сконфигурированной сети;
- организовать точку доступа Wi-Fi 2.4 GHz либо Ethernet для подключения ПЛК;
- привязать IP адреса ПЛК контроллеров к доменным именам, что удобно обращаться к устройству из фреймворка просмотреть в какое время было подключено устройство, принудительно отключить устройство от Wi-Fi сети Hub;

Служба RasAP позволяет через web интерфейс в динамике видеть какие устройства сейчас подключены, какой IP адрес был выдан, сколько времени устройство на

Работа пользователя в части разработки серверного прикладного кода проходит в комфортной среде потокового программирования Node-Red, на языке JavaScript, при этом возможна одновременная разработка проекта на нескольких языках в том числе на Python.

Программный комплекс предоставляет мощные, средства построения верхнего уровня систем автоматизации. Программно-технический комплекс функционально и гибко стыкуется с промышленными системами класса MES, APS, SCADA.

Платформа "Horizon Automation Server" идеально подходит для организации разнообразных образовательных программ для различного уровня учебных заведений, по курсам - физика, химия, информатика, математическое моделирование, робототехника, моделирование БПЛА и др.